EduLens

NLP and Machine Learning-Based Student Learning Intelligence and Performance Analytics Platform

---

1. Project Overview

EduLens is a production-ready, Dockerized student learning intelligence and performance analytics platform.

The core idea is that students can learn any topic using external Large Language Model (LLM) clients such as **ChatGPT, Claude, or other MCP-compatible LLM clients**. EduLens is not primarily the tutor. Instead, it acts as an **intelligence and analytics layer** that analyzes the student's learning conversations and assessment performance over time.

The student first learns a topic through an external LLM client. After completing a learning session, the student instructs the LLM client to transfer the conversation to EduLens.

The LLM client communicates with the **EduLens MCP Server**, which receives the relevant conversation context. EduLens then automatically processes the raw conversation data, extracts learning-related information using NLP and Data Science techniques, generates a targeted quiz using the Gemini API, collects the student's quiz performance, and eventually uses the combined conversational and assessment data for machine learning, prediction, analytics, and personalized reports.

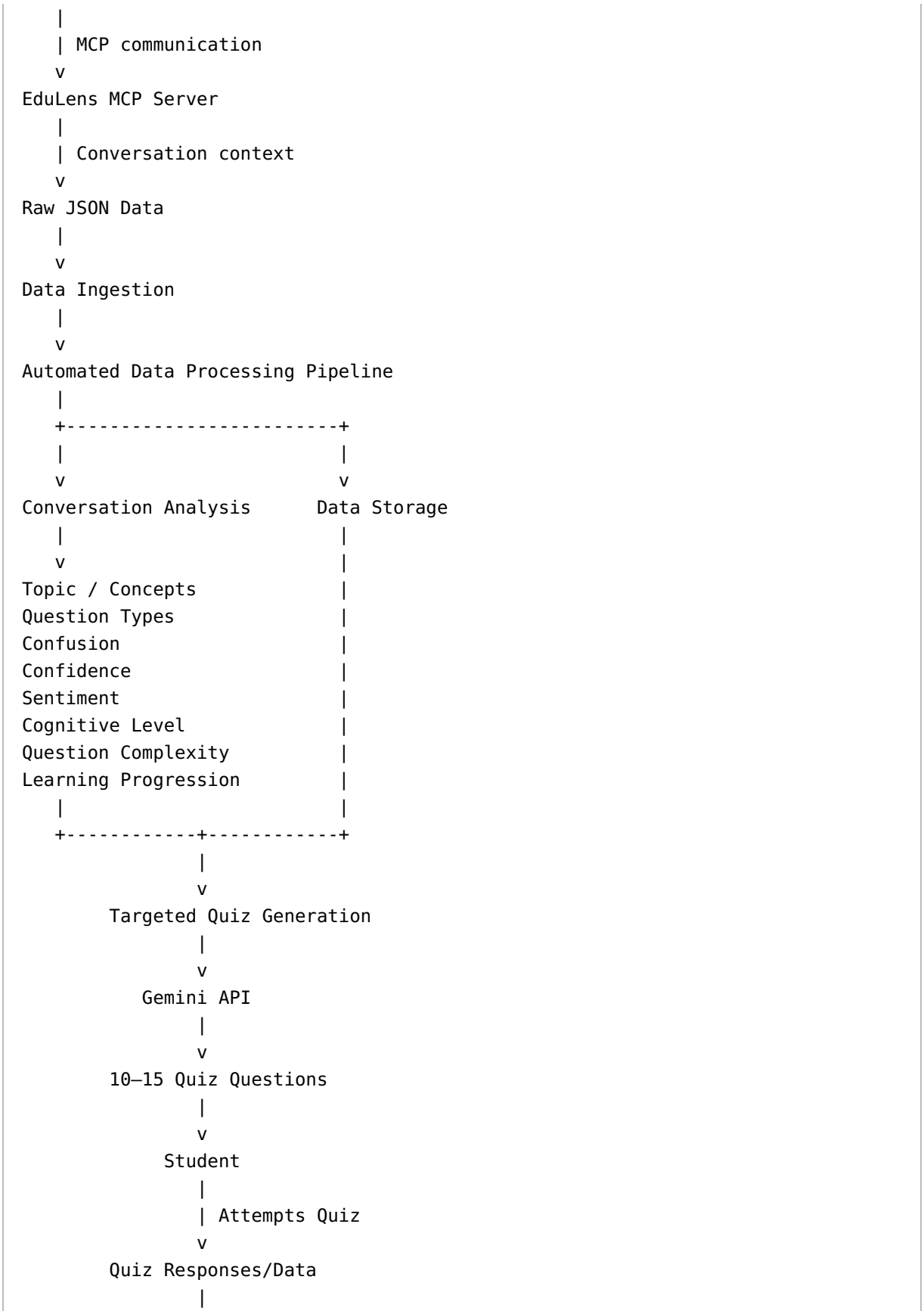
The final application should be designed as a **production-ready, modular, scalable, Dockerized application**, not as a simple college demonstration or collection of notebooks.

---

2. Core Project Flow

The complete high-level flow is:

```
Student
   |
   | Learns a topic
   v
ChatGPT / Claude / Other LLM
   |
   | "Send this learning session to EduLens"
   v
LLM Client
```


|
| MCP communication
v
EduLens MCP Server
|
| Conversation context
v
Raw JSON Data
|
v
Data Ingestion
|
v
Automated Data Processing Pipeline
|
+-------------------------+
| |
v   v
Conversation Analysis   Data Storage
| |
v |
Topic / Concepts |
Question Types |
Confusion |
Confidence |
Sentiment |
Cognitive Level |
Question Complexity |
Learning Progression |
| |
+------------+------------+
|
v
Targeted Quiz Generation
|
v
Gemini API
|
v
10–15 Quiz Questions
|
v
Student
|
| Attempts Quiz
v
Quiz Responses/Data
|

```
          v
   Combined Learning Data
          |
          v
   Data Preprocessing
          |
          v
   Feature Engineering
          |
          v
      NLP / Data Science
          |
          v
      ML Model Pipeline
          |
          v
   Student Learning Model
          |
          v
  Predictions + Analytics
          |
          v
   Reports / Dashboard
```

---

3. Important Architectural Principle

EduLens should **not** rely on an LLM agent to perform every Data Science operation.

The system should use:

> **Deterministic code for data processing + ML models for prediction + LLMs where generative/reasoning capabilities are actually useful.**

For example:

Conventional Python/Data Science code

Responsible for:

- JSON parsing
- validation
- cleaning
- preprocessing
- feature engineering

- statistical calculations
- database operations
- model inference
- metric calculation
- data aggregation

NLP models

Responsible for:

- topic extraction
- concept extraction
- sentiment/emotion analysis
- confusion detection
- question classification
- cognitive-level estimation
- semantic analysis
- embeddings

Machine Learning

Responsible for:

- mastery prediction
- performance prediction
- learning-state estimation
- identifying patterns
- risk prediction
- personalized analytics

Gemini API

Responsible primarily for:

- generating targeted quiz questions
- generating questions based on extracted topics/concepts
- adapting questions to identified weak areas and appropriate difficulty

MCP

Responsible for:

- communication between the external LLM client and EduLens
- exposing EduLens functionality as MCP tools
- receiving the learning-session context from the LLM client

---

4. MCP Integration

EduLens will contain an **MCP Server**.

The external LLM client acts as the MCP client.

Example interaction:

```
Student:
"Send this learning session to EduLens."
```

The LLM client invokes an EduLens MCP tool such as:

```
export_learning_session
```

The tool sends relevant conversation data to EduLens.

Conceptually:

```
LLM Client
     |
     | MCP
     v
EduLens MCP Server
     |
     v
EduLens Backend
```

The MCP server should not be responsible for performing the complete Data Science pipeline.

Its primary responsibility is to receive/validate the learning-session payload and hand it to the EduLens backend.

---

5. Raw Conversation Data

The conversation transferred through MCP should be normalized into a structured JSON representation.

Example:

```
{
  "student_id": "student_001",
  "session_id": "session_001",
  "source": "chatgpt",
  "topic": null,
  "started_at": "2026-09-08T09:30:00",
  "ended_at": "2026-09-08T10:15:00",
  "messages": [
    {
      "role": "user",
      "content": "Explain binary search to me.",
      "timestamp": "2026-09-08T09:30:10"
    },
    {
      "role": "assistant",
      "content": "Binary search is an algorithm...",
      "timestamp": "2026-09-08T09:30:15"
    },
    {
      "role": "user",
      "content": "I don't understand why the array needs to be sorted.",
      "timestamp": "2026-09-08T09:32:00"
    }
  ]
}
```

The exact schema should be finalized during implementation.

The system should support different LLM sources through a **source-agnostic conversation adapter/ normalization layer**, rather than making the entire pipeline dependent on ChatGPT-specific formatting.

---

6. Conversation Analysis

After receiving the raw JSON, EduLens automatically analyzes the conversation.

The system should primarily analyze the **student's messages**, while using assistant messages as contextual information.

Potential extracted signals include:

Topic

Example:

```
Data Structures
```

Main concepts

```
Binary Search
Sorted Arrays
Divide and Conquer
Time Complexity
```

Question type

Examples:

```
Conceptual
Application
Debugging
Definition
Why/How
Problem Solving
```

Confusion

Estimate whether the student appears confused about a particular concept.

Confidence

Estimate the student's apparent confidence based on conversational signals.

Sentiment / emotional state

Sentiment should be treated as **one learning signal**, not the central purpose of the project.

Cognitive level

Potential categories:

```
Remember
Understand
Apply
Analyze
Evaluate
Create
```

Question complexity

Estimate the semantic/technical complexity of the questions being asked.

Misconceptions

Identify concepts where the student's reasoning repeatedly appears incorrect.

Learning progression

Track whether the student's questions and understanding appear to improve during the session.

---

7. Topic Extraction

After receiving the conversation, the system should determine:

1. Main topic
2. Subtopics
3. Important concepts
4. Potential weak concepts

For example:

```
Main Topic:
Binary Search

Subtopics:
- Sorted arrays
- Search space reduction
- Midpoint calculation
- Time complexity

Potential Weak Areas:
- Why sorting is required
- How the search space is reduced
```

These extracted concepts will later be used to generate a targeted quiz.

---

8. Quiz Generation

After conversation analysis, EduLens should provide the student with an option to attempt a quiz.

The system will use the **Gemini API** to generate approximately **10–15 questions** based on the actual learning conversation.

The questions should not simply be random questions about the general topic.

They should be targeted using information such as:

```
Main topic
Subtopics
Weak concepts
Confusion
Question difficulty
Learning level
Conversation-derived features
```

For example:

```
Student studied:
Binary Search

Detected weakness:
Understanding why the array must be sorted.

Quiz generation should therefore include:
- basic binary search questions
- conceptual questions
- application questions
- questions specifically testing the identified weakness
```

The generated quiz should be stored so that the exact questions presented to the student can later be analyzed.

---

9. Quiz Data Collection

When the student attempts the quiz, EduLens should collect the complete assessment information.

Potential fields:

```
student_id
session_id
quiz_id
question_id
```

```
topic
concept
difficulty
question
correct_answer
student_answer
is_correct
time_taken
attempt_timestamp
```

If appropriate, the system can also collect:

```
confidence before answering
number of attempts
question-level performance
```

This creates structured assessment data that can be combined with the conversation-derived features.

---

10. Combined Data Pipeline

EduLens should combine:

Conversational data

```
Topics
Concepts
Questions
Confusion
Confidence
Sentiment
Cognitive level
Question complexity
Learning progression
Misconceptions
Engagement
```

with:

Quiz data

```
Questions attempted
Correct answers
```

```
Incorrect answers
Accuracy
Difficulty
Time taken
Topic-wise performance
Concept-wise performance
```

and potentially:

Behavioral data

```
Study duration
Number of sessions
Session frequency
Learning consistency
Repeated topics
Revision frequency
```

This combined dataset becomes the foundation of the Data Science and Machine Learning layer.

---

11. Data Preprocessing

The preprocessing pipeline should be automated.

The team should write deterministic Python functions once, and those functions should automatically execute whenever new data enters the system.

Example:

```
Raw JSON
   ↓
Validation
   ↓
Cleaning
   ↓
Normalization
   ↓
Missing-value handling
   ↓
Duplicate detection
   ↓
Text preprocessing
   ↓
```

```
Feature extraction
   ↓
Structured dataset
```

The system should not require manually running a notebook whenever a new learning session arrives.

---

12. Automated Data Pipeline

The production system should operate approximately as follows:

```
New Learning Session
        ↓
MCP Server
        ↓
FastAPI Backend
        ↓
Data Ingestion
        ↓
Queue / Background Job
        ↓
Processing Worker
        ↓
Preprocessing
        ↓
NLP Analysis
        ↓
Feature Extraction
        ↓
Database
        ↓
Student Model Update
```

For the initial implementation, a simple background-job architecture can be used.

The system does not need Kafka or a highly complex distributed architecture from day one.

---

13. Data Science Pipeline

The Data Science layer should include:

Exploratory Data Analysis

Analyze:

- study patterns
- quiz performance
- topic performance
- question complexity
- confusion patterns
- learning consistency
- relationships between conversational signals and assessment results

Feature Engineering

Potential features:

```
average_confusion
average_confidence
question_complexity
study_duration
session_frequency
quiz_accuracy
concept_mastery
topic_repetition
question_count
learning_progression
sentiment_statistics
```

Statistical Analysis

Investigate relationships such as:

```
Does higher confusion predict lower quiz performance?

Does question complexity increase as mastery improves?

Does learning consistency correlate with performance?

Do conversational features provide predictive information
beyond quiz scores and study time?
```

---

14. Machine Learning

The final system should contain functional ML models.

Possible prediction targets include:

Topic Mastery

Estimate:

```
Probability that the student has mastered a topic
```

Example:

```
Binary Search Mastery: 0.78
```

Performance Prediction

Predict expected performance on a future assessment.

Learning Risk

Identify whether a student may be struggling or falling behind.

Improvement Prediction

Estimate whether the student's performance is improving after additional learning.

Next Learning Recommendation

Identify:

```
What should the student revise next?
Which concept needs more attention?
What difficulty level should the next quiz use?
```

The exact ML targets should be finalized after sufficient data becomes available.

---

15. Important ML Development Constraint

The project should not claim that a sophisticated predictive model can be trained immediately without sufficient data.

During the early development stage:

```
Prototype Data
      ↓
Feature Engineering
      ↓
Baseline ML Models
      ↓
Evaluation
      ↓
Real User Data Collection
      ↓
Improved Models
```

Potential initial models include:

```
Linear Regression
Logistic Regression
Random Forest
XGBoost
Gradient Boosting
```

The final model should be selected based on actual evaluation rather than assuming one algorithm is best.

---

16. Longitudinal Student Model

A major feature of EduLens is that it should not analyze every session independently.

The system should maintain a continuously updated representation of the student's learning state.

Example:

```
Student: 001

Topic: Binary Search
```

```
Mastery: 0.72
Confidence: 0.68
Confusion: 0.31
Quiz Accuracy: 80%
Sessions: 5
Last Studied: 2026-09-08
```

After another session:

```
Mastery: 0.79
Confidence: 0.75
Confusion: 0.22
Quiz Accuracy: 87%
```

This allows EduLens to perform **longitudinal learning analysis**.

---

17. Reports and Dashboard

The frontend should present useful learning intelligence rather than simply displaying raw data.

Possible dashboard sections:

Overview

```
Overall learning progress
Current mastery
Recent activity
Quiz performance
```

Topic Analysis

```
Strong Topics
Weak Topics
Topics improving
Topics requiring revision
```

Conversation Intelligence

```
Question complexity
Confusion trends
```

```
Confidence trends
Learning progression
```

Assessment Analytics

```
Quiz accuracy
Topic-wise accuracy
Concept-wise accuracy
Difficulty-wise performance
```

Prediction

```
Mastery prediction
Performance prediction
Learning risk
```

Recommendations

```
Revise Binary Search midpoint calculation.

Practice medium-level application questions.

Review why binary search requires a sorted search space.
```

---

18. Before vs After Learning Analysis

One of the strongest analytical features should be comparison across a learning period.

Example:

```
Before Learning

Binary Search Mastery: 0.38
Confusion: 0.71
Quiz Accuracy: 40%

        ↓

Learning Session
```

```
        ↓

After Learning

Binary Search Mastery: 0.81
Confusion: 0.24
Quiz Accuracy: 85%
```

This allows EduLens to demonstrate measurable learning progression.

---

19. Suggested Database Structure

Use **PostgreSQL** as the primary relational database.

Logical tables:

```
students
-------------------------
student_id
created_at


learning_sessions
-------------------------
session_id
student_id
source
topic
started_at
ended_at
duration


messages
-------------------------
message_id
session_id
role
content
timestamp


message_analysis
-------------------------
```

```
message_id
topic
concept
sentiment
confusion
confidence
question_type
cognitive_level
complexity


quiz_attempts
-------------------------
attempt_id
student_id
quiz_id
question_id
topic
concept
difficulty
student_answer
correct
time_taken
timestamp


student_topic_state
-------------------------
student_id
topic
mastery_score
confidence
confusion
last_updated


daily_student_metrics
-------------------------
student_id
date
study_time
sessions
questions
quiz_accuracy
average_confusion
average_sentiment
mastery_change
```

The exact database schema can evolve during implementation.

---

20. Technology Stack

Frontend

Recommended:

```
React
Next.js
TypeScript
Tailwind CSS
Plotly / Recharts
```

The frontend should provide:

- dashboard
- quiz interface
- analytics
- reports
- student progress
- topic analysis

Backend

Recommended:

```
Python
FastAPI
Pydantic
SQLAlchemy
```

FastAPI should provide:

- REST APIs
- authentication-related endpoints
- quiz endpoints
- analytics endpoints
- report endpoints
- MCP integration
- data ingestion

MCP

Recommended:

```
Python MCP SDK
```

The MCP server should expose tools such as:

```
export_learning_session
get_student_progress
get_weak_topics
get_learning_report
get_student_learning_state
```

The initial MVP should focus primarily on:

```
export_learning_session
```

Additional tools can be added later.

NLP

Recommended:

```
Python
Hugging Face Transformers
spaCy
sentence-transformers
NLTK (optional)
```

Potential NLP tasks:

```
Topic extraction
Concept extraction
Sentiment analysis
Confusion detection
Question classification
Semantic similarity
```

```
Embeddings
Text classification
```

Data Science

Recommended:

```
Pandas
NumPy
SciPy
Matplotlib
Plotly
```

Machine Learning

Recommended:

```
scikit-learn
XGBoost
Joblib
```

Potential future additions:

```
PyTorch
TensorFlow
Knowledge tracing models
Sequence models
```

These should only be added if justified by the data and research requirements.

Generative AI

Use:

```
Gemini API
```

Primary responsibility:

```
Targeted quiz generation
```

The Gemini API should receive structured information such as:

```
topic
concepts
weak concepts
difficulty
question types
learning signals
```

rather than blindly receiving the entire raw database.

Database

```
PostgreSQL
```

Optional:

```
Redis
```

Redis can initially be used for:

- background jobs
- caching
- queue support

Background Processing

Possible initial implementation:

```
Celery
Redis
```

or another lightweight background task mechanism.

The architecture should allow the processing system to scale later.

Deployment

The entire application should be **Dockerized**.

Potential containers:

```
frontend
backend
mcp-server
worker
postgres
redis
```

For local development:

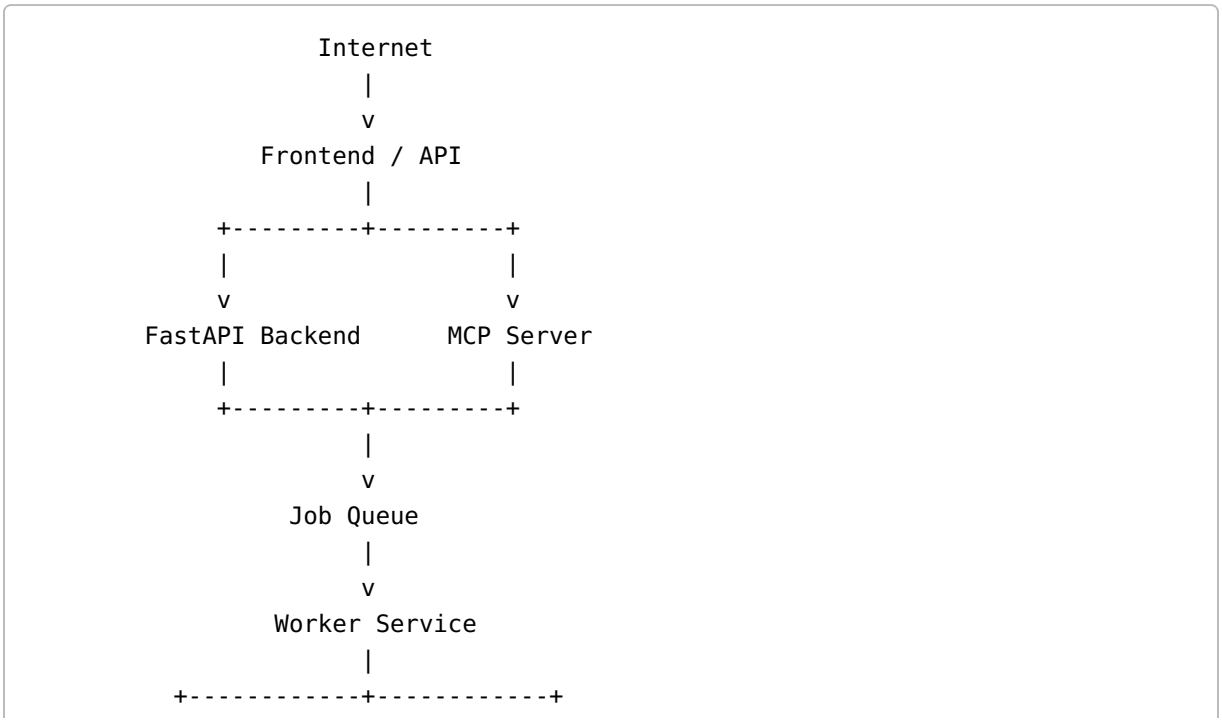
```
Docker Compose
```

should be used.

---

21. Dockerized Production Architecture

The target architecture should look approximately like:



```
          |            |            |
          v            v            v
         NLP          ML       Data Processing
          |            |            |
          +------------+------------+
                       |
                       v
                PostgreSQL
                       |
                       v
                Dashboard
```

---

22. Recommended Project Structure

A starting repository structure:

```
edulens/
│
├── frontend/
│   ├── app/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── hooks/
│   ├── types/
│   ├── package.json
│   └── Dockerfile
│
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   │
│   │   ├── api/
│   │   │   ├── routes/
│   │   │   │   ├── sessions.py
│   │   │   │   ├── quizzes.py
│   │   │   │   ├── analytics.py
│   │   │   │   └── reports.py
│   │   │   │
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   └── security.py
│   │   │
│   │   ├── models/
```

```
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── database/
│   │   └── utils/
│   │
│   ├── requirements.txt
│   └── Dockerfile
│
├── mcp_server/
│   ├── server.py
│   ├── tools/
│   │   └── learning_session.py
│   ├── schemas/
│   ├── adapters/
│   ├── requirements.txt
│   └── Dockerfile
│
├── data_pipeline/
│   ├── ingestion/
│   ├── validation/
│   ├── preprocessing/
│   ├── feature_engineering/
│   ├── aggregation/
│   └── pipeline.py
│
├── nlp/
│   ├── topic_extraction.py
│   ├── concept_extraction.py
│   ├── sentiment.py
│   ├── confusion_detection.py
│   ├── question_classification.py
│   ├── embeddings.py
│   └── analyzer.py
│
├── ml/
│   ├── training/
│   ├── inference/
│   ├── preprocessing/
│   ├── features/
│   ├── evaluation/
│   ├── models/
│   └── prediction.py
│
├── quiz/
│   ├── generator.py
│   ├── evaluator.py
│   ├── schemas.py
│   └── gemini_client.py
```

```
│
├── workers/
│   ├── tasks.py
│   └── worker.py
│
├── analytics/
│   ├── daily_metrics.py
│   ├── weekly_analysis.py
│   ├── trends.py
│   └── reports.py
│
├── tests/
│   ├── test_api/
│   ├── test_mcp/
│   ├── test_pipeline/
│   ├── test_nlp/
│   └── test_ml/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── sample/
│
├── notebooks/
│   ├── eda/
│   └── research/
│
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

This is a starting structure, not a requirement that every directory must exist on day one.

---

23. Important Development Rule

Do not build the entire project simultaneously.

Development should proceed incrementally.

Recommended order:

```
Phase 1
Basic backend + database
        ↓
Phase 2
MCP integration
        ↓
Phase 3
Raw JSON ingestion
        ↓
Phase 4
Automated preprocessing pipeline
        ↓
Phase 5
NLP analysis
        ↓
Phase 6
Quiz generation through Gemini
        ↓
Phase 7
Quiz UI + result collection
        ↓
Phase 8
Feature engineering
        ↓
Phase 9
ML baseline models
        ↓
Phase 10
Student learning-state model
        ↓
Phase 11
Analytics + reports
        ↓
Phase 12
Dockerization
        ↓
Phase 13
Testing + security + production hardening
```

---

24. Production-Ready Requirements

EduLens should not be treated as a notebook-based academic prototype.

The final application should include:

Containerization

```
Docker
Docker Compose
```

Configuration

Use environment variables for:

```
DATABASE_URL
GEMINI_API_KEY
SECRET_KEY
API configuration
```

Never hardcode API keys.

Logging

Implement structured application logging.

Error handling

The system should gracefully handle:

- malformed JSON
- invalid sessions
- missing fields
- API failures
- Gemini API failures
- NLP failures
- database failures
- duplicate sessions

Validation

Use Pydantic schemas and validation at API/MCP boundaries.

Testing

Include:

```
Unit tests
Integration tests
API tests
```

```
MCP tests
Pipeline tests
ML evaluation tests
```

Database migrations

Use:

```
Alembic
```

with PostgreSQL.

API documentation

FastAPI's OpenAPI documentation should be maintained.

Security

At minimum:

- authentication
- authorization
- input validation
- secure environment variables
- database security
- protection of student data
- proper handling of conversation data

---

25. Data Privacy

Conversation data can contain sensitive personal information.

EduLens should therefore:

- collect only necessary data
- clearly define what is stored
- avoid unnecessary personal information
- secure stored conversation data
- protect API credentials
- use authentication and authorization
- provide appropriate data deletion mechanisms

The system should not assume that an LLM client gives EduLens unrestricted access to a user's conversation history.

The user/client must explicitly provide the relevant conversation context through the supported integration.

---

26. Research Component

EduLens should have a meaningful Data Science/ML research component.

A potential experiment:

Model A — Traditional Features

Use:

```
study time
quiz scores
previous performance
session frequency
```

Model B — Extended Learning Intelligence Features

Use:

```
traditional features
+
confusion
+
confidence
+
question complexity
+
concept progression
+
conversation-derived features
+
learning-state features
```

Then compare the models using metrics such as:

```
MAE
RMSE
R²
Accuracy
Precision
```

```
Recall
F1
ROC-AUC
```

depending on the prediction task.

The research question becomes:

> Do conversational learning signals provide additional predictive value beyond traditional academic and behavioral features?

This gives EduLens a stronger academic/research justification.

---

27. Example End-to-End Scenario

A student wants to learn:

```
Binary Search
```

They open ChatGPT and have a conversation:

```
Student:
Explain binary search.

ChatGPT:
...

Student:
Why does the array need to be sorted?

ChatGPT:
...

Student:
I still don't understand how the search space is reduced.

ChatGPT:
...
```

The student then says:

```
Send this learning session to EduLens.
```

The LLM client sends the relevant conversation context through MCP.

EduLens receives:

```
Raw JSON
```

The pipeline automatically processes it.

EduLens identifies:

```
Topic:
Binary Search

Weak Concept:
Search-space reduction

Confusion:
High

Question Complexity:
Medium

Learning Level:
Understanding
```

The student clicks:

```
Attempt Quiz
```

Gemini generates 10–15 targeted questions.

The student completes the quiz.

Example:

```
Score: 7/10
Accuracy: 70%

Weak Concept:
```

```
Search-space reduction

Strong Concept:
Basic binary search procedure
```

The system combines:

```
Conversation Data
+
Quiz Data
+
Historical Student Data
```

Then updates:

```
Binary Search Mastery: 0.68
Confusion: 0.35
Confidence: 0.61
```

The ML system generates predictions and the dashboard displays:

```
Binary Search
-------------------------
Mastery       68%
Quiz Score    70%
Confidence    61%
Confusion     35%

Recommendation:
Revise search-space reduction
and attempt another medium-level quiz.
```

Over multiple sessions, EduLens builds a longitudinal model of the student's learning behavior and performance.

---

28. Final Product Definition

EduLens should ultimately be understood as:

> **A learning intelligence platform that connects to external LLM-based learning environments through MCP, automatically processes learning conversations and**

assessment data, applies NLP and Machine Learning to model student learning states, generates targeted assessments, and provides longitudinal performance analytics, predictions, and personalized learning insights.

The project combines:

```
MCP
+
LLM Integration
+
NLP
+
Data Science
+
Machine Learning
+
Generative AI
+
Backend Engineering
+
Database Engineering
+
Data Pipelines
+
Visualization
+
Docker
+
Production Software Engineering
```

The primary objective is not to build another AI tutor.

The primary objective is to build the **intelligence layer around the student's existing AI-assisted learning process**.

---

29. One-Sentence Architecture

```
External LLM → MCP → EduLens Ingestion → Automated Data Pipeline → NLP → Feature Engineering → Quiz Generation → Quiz Data → ML → Student Learning Model → Predictions → Analytics Dashboard & Reports
```

---

30. Key Implementation Philosophy

The development team/LLM assisting with this project should follow these principles:

1. **Build modularly.**
2. **Keep deterministic data processing in code.**
3. **Use LLMs only where generative/reasoning capabilities add value.**
4. **Use ML models for measurable predictive tasks.**
5. **Keep MCP as the integration/interface layer.**
6. **Automate the data pipeline so new data does not require manual execution.**
7. **Store raw and processed data separately.**
8. **Maintain reproducible preprocessing and ML pipelines.**
9. **Do not over-engineer the MVP.**
10. **Design for production from the beginning.**
11. **Dockerize the complete application.**
12. **Write tests alongside the implementation.**
13. **Treat student conversation data as private data.**
14. **Do not make unsupported claims about prediction accuracy before evaluating the models.**
15. **Prioritize actual working functionality over adding unnecessary AI agents.**

Final MVP

The minimum successful version of EduLens should be able to perform this complete cycle:

```
1. Student learns through ChatGPT/Claude
                ↓
2. Student requests transfer to EduLens
                ↓
3. LLM client sends conversation through MCP
                ↓
4. EduLens stores raw conversation as structured JSON
                ↓
5. Automated pipeline processes the conversation
                ↓
6. NLP extracts topic, concepts and learning signals
                ↓
7. Gemini generates 10–15 targeted quiz questions
                ↓
8. Student attempts quiz
                ↓
9. Quiz responses are stored
                ↓
10. Conversation + quiz data are combined
                ↓
```

```
11. Data preprocessing + feature engineering
                ↓
12. ML model processes the student's learning data
                ↓
13. Student learning state is updated
                ↓
14. Predictions and analytics are generated
                ↓
15. Dashboard/report presents the results
```

The final system must be **functional, tested, modular, deployable, and Dockerized**.